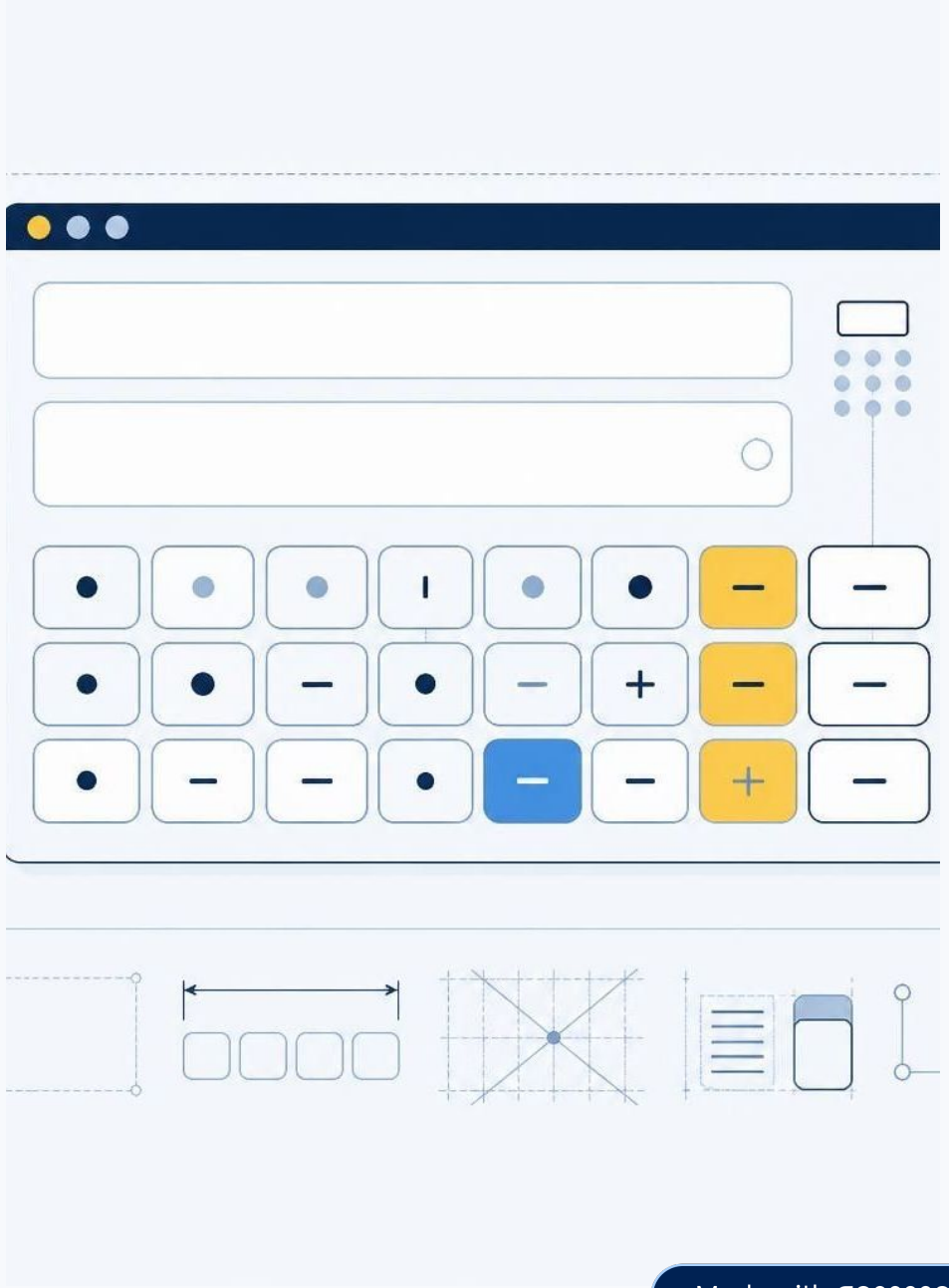
PYTHON DE CERO A GUI

Interfaces gráficas con Python y Tkinter

Construye aplicaciones de escritorio visuales e intuitivas sin necesidad de librerías externas.

tkinter · Biblioteca estándar incluida con Python



Made with GAMMA

PARADIGMA VISUAL

Una GUI permite interactuar sin usar la consola

En lugar de escribir comandos en una terminal negra, el usuario interactúa con controles gráficos: ventanas, botones, textos y campos de entrada.

Acciones visibles

Cada operación tiene una representación en pantalla que reacciona de inmediato al clic.

Nativo en Python

Tkinter permite crear aplicaciones de escritorio y viene preinstalado en Python para Windows, macOS y Linux sin requerir `pip install`.

TERMINAL VS VENTANA

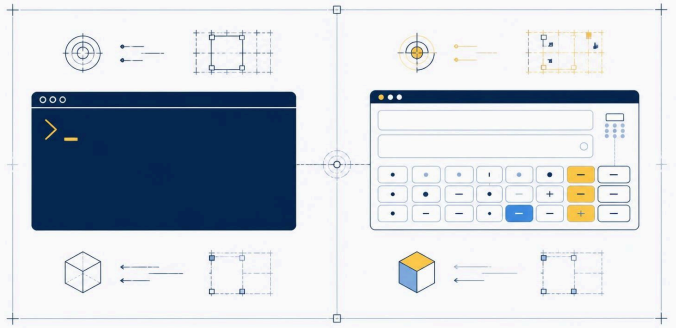
Consola:

```
num = input("Número: ")
```

Interfaz Tkinter:

```
caja = tk.Entry(ventana)
```

```
btn = tk.Button(text="OK")
```



Made with GAMMA

ESTRUCTURA BASE · PYTHON 3 · TKINTER

La ventana principal es el punto de partida

La clase `Tk()` crea la ventana que contiene los widgets. `title()` define el título; `geometry()` define tamaño y posición; `resizable()` permite o impide cambiar el tamaño.

```
import tkinter as tk; ventana = tk.Tk()
ventana.title("Calculadora"); ventana.geometry("320x420")
ventana.resizable(False, False)
ventana.mainloop()
```

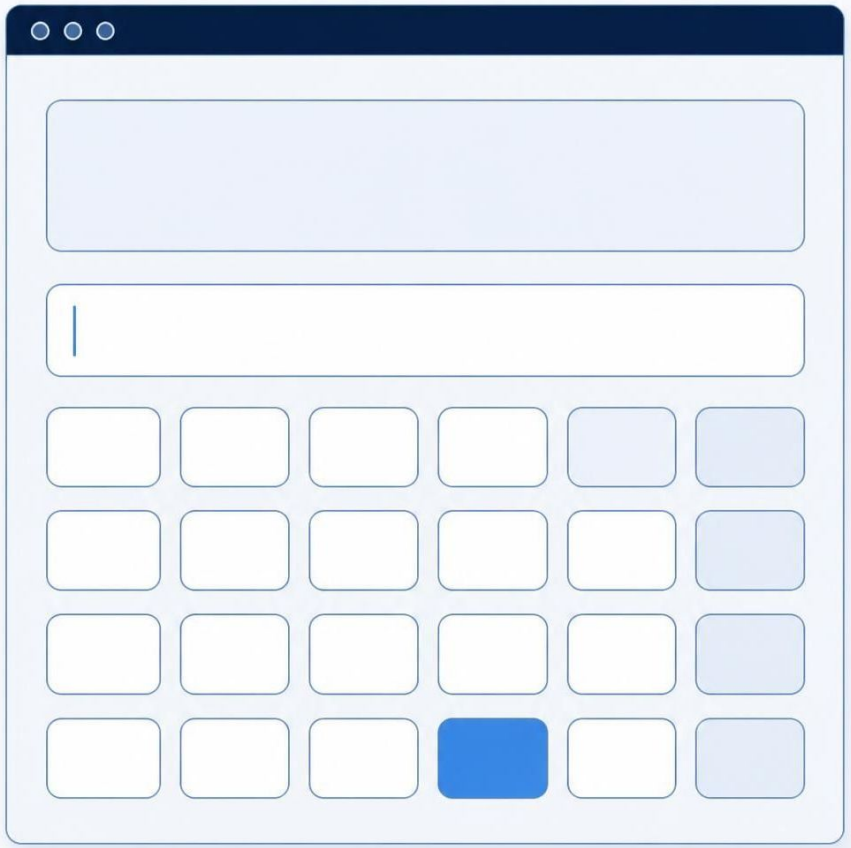
En este ejemplo: 320 × 420 píxeles; el redimensionamiento manual está bloqueado.

Made with GAMMA

COMPONENTES CLAVE

Label muestra información y Entry recibe datos

Las etiquetas informan y las cajas de entrada capturan texto o cifras del teclado.



tk.Label

Muestra títulos, instrucciones y resultados, como texto estático o dinámico.

tk.Entry

Permite escribir números o palabras; get() recupera el contenido como texto.

Propiedades visuales: `font=("Arial", 14)` define letra y tamaño; `justify="right"` alinea los números a la derecha.

Made with GAMMA

PROGRAMACIÓN POR EVENTOS

Los botones conectan la interfaz con funciones

Button crea una acción; command vincula la función sin ejecutarla: se pasa su nombre sin paréntesis.

```
def sumar(): print("Calculando suma...")
btn = tk.Button(ventana, text="SUMAR",
                command=sumar, width=12)
btn.pack(pady=6)
```

SUMAR

muestra la suma de las dos entradas

RESTAR

calcula su diferencia

LIMPIAR

vacía los campos

SALIR

cierra la ventana

Made with GAMMA

GEOMETRY MANAGERS

pack() organiza los elementos en la ventana

Tkinter requiere colocar explícitamente cada widget en el lienzo. `pack()` apila los elementos en orden vertical de arriba hacia abajo.

Parámetros esenciales

pady: Agrega margen vertical externo para evitar que los controles queden amontonados.

width: Especifica el ancho en caracteres para equilibrar botones y entradas.

ORDEN DE APILADO CON PACK()

1. **tk.Label(text="Primer número").pack(pady=4)**
2. **entry1.pack(pady=4)** · Campo de texto
3. **btn_sumar.pack(pady=6)** · Botón de acción
4. **lbl_resultado.pack(pady=8)** · Salida final

Made with GAMMA

LÓGICA MATEMÁTICA

Sumar y restar requieren leer y convertir datos

El texto que ingresa el usuario debe convertirse a tipo numérico (`float` o `int`) antes de calcular.

01

1. Obtener texto

`val = entry.get()` extrae el string.

02

2. Convertir y operar

`float(val)` permite sumar o restar.

03

3. Actualizar pantalla

`config(text=res)` refresca la etiqueta.

FUNCIÓN SUMAR CON CONTROL DE ERRORES

```
def sumar():
    try:
        n1 = float(entry1.get())
        n2 = float(entry2.get())
        res = n1 + n2
        lbl_resultado.config(
            text=f"Resultado: {res}"
        )
    except ValueError:
        lbl_resultado.config(
            text="¡Error! Ingrese números"
        )
```

Made with GAMMA

ESTADOS Y CIERRE

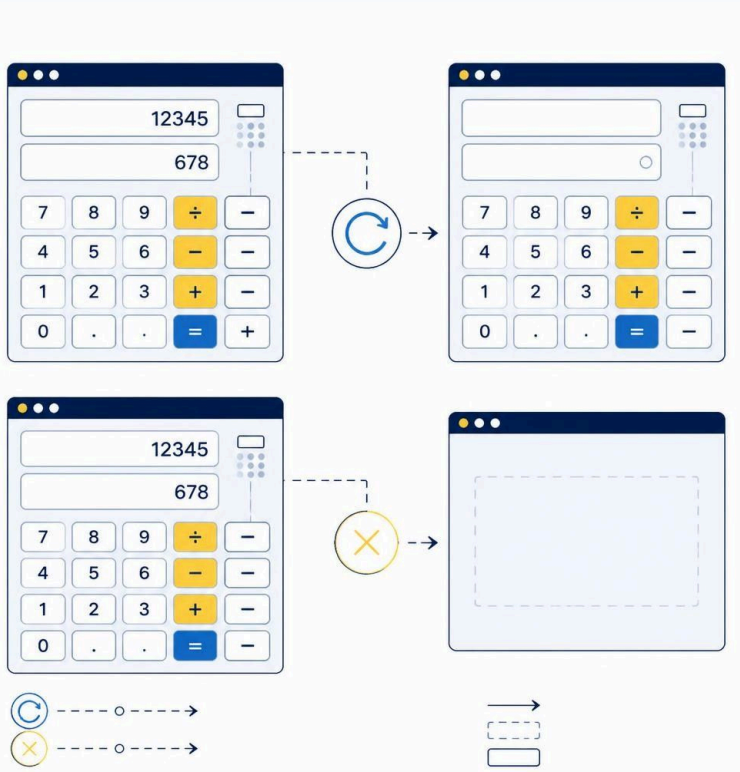
Limpiar y salir completan la interacción

Reiniciar los campos y cerrar el programa completa la experiencia de usuario.

`entry.delete(0, tk.END)`
Borra todo el texto del campo, desde el índice 0 hasta el final.

`lbl_resultado.config(text="Resultado: ")`
Devuelve la etiqueta de respuesta a su estado neutro original.

`ventana.destroy()`
Cierra la ventana de forma limpia y libera los recursos del sistema.



Made with GAMMA

MOTOR DE EJECUCIÓN

mainloop() mantiene activa la interfaz

El ciclo procesa eventos del sistema operativo: clics, pulsaciones de teclas y redibujado de la pantalla.



Crear ventana: tk.Tk() inicializa el contexto.



Definir lógica: funciones sumar y restar.



Ubicar widgets: pack() organiza los controles.



Ejecutar mainloop(): espera y atiende eventos.

Regla fundamental: el código después de `mainloop()` no se ejecuta hasta que el usuario cierra la ventana.

Made with GAMMA

SÍNTESIS FINAL

Tkinter construye la interfaz; Python define su comportamiento

La GUI separa la presentación visual de la lógica matemática del programa.

Ventana

Tk() · title()
geometry() · resizable()

Configuración del contenedor principal.

Widgets y diseño

Label() · Entry()
Button() · pack()
font · pady · width · justify

Elementos visibles y disposición visual.

Interacción

get() · config()
delete() · mainloop()
destroy()

Lectura de datos, eventos y ciclo de vida de la app.

Made with GAMMA